

## Resolución: Análisis del Algoritmo GeoGreedy (Enrutado Voraz Geográfico)

Este documento analiza el comportamiento del algoritmo voraz GeoGreedy, detalla las estructuras de datos óptimas para su implementación y demuestra si la estrategia encuentra o no la solución óptima mediante contraejemplos formales.

### 1. Estructura de Datos Adecuada

Para implementar el algoritmo GeoGreedy de forma eficiente, necesitamos una estructura que permita representar la topología del grafo y las coordenadas espaciales de cada nodo.

#### Componentes de la Estructura

1. **Grafo mediante Lista de Adyacencia:** Es la representación más eficiente porque en cada paso el algoritmo solo necesita examinar los **nodos adyacentes** al nodo actual  $u$ . Una lista de adyacencia permite iterar sobre los vecinos del nodo actual en tiempo  $O(\text{grado}(u))$ , evitando tener que recorrer nodos no conectados.
2. **Diccionario / Tabla de Hash de Coordenadas:** Cada nodo  $i$  debe estar asociado a sus coordenadas geográficas  $(x_i, y_i)$  en el plano bidimensional. Un diccionario permite obtener las coordenadas de cualquier nodo en tiempo constante  $O(1)$ .

#### Funcionamiento del paso voraz con estas estructuras

Cuando el mensaje llega al nodo  $u$ :

1. Se accede a la lista de vecinos de  $u$  en tiempo  $O(1)$ .
2. Para cada vecino  $v$ , se buscan sus coordenadas en la tabla de hash ( $O(1)$ ) y se calcula la distancia euclídea al objetivo  $o$ :

$$d(v, o) = \sqrt{(x_v - x_o)^2 + (y_v - y_o)^2}$$

3. Se selecciona el vecino con la distancia  $d(v, o)$  mínima. Este escaneo toma un tiempo lineal respecto al número de vecinos:  $O(\text{grado}(u))$ .

### 2. ¿Encuentra esta estrategia voraz la solución óptima?

La respuesta es **NO**. La estrategia voraz GeoGreedy no garantiza encontrar la solución óptima (ni en número de saltos ni en distancia recorrida).

De hecho, presenta dos problemas fundamentales:

1. **Incompletitud (Mínimos Locales / "Voids"):** El algoritmo puede quedarse atrapado en un nodo intermedio que está más cerca del objetivo que todos sus vecinos, finalizando el proceso sin llegar a entregar el mensaje.

2. **Suboptimalidad:** Incluso si logra llegar al objetivo, la ruta elegida puede ser significativamente más larga (tanto en distancia como en número de saltos) que la ruta óptima.

### 3. Contraejemplos

#### Contraejemplo 1: Fallo absoluto por Mínimo Local (El algoritmo se queda atrapado)

Supongamos que queremos enviar un mensaje desde el nodo inicial  $i$  al objetivo  $o$ . Las coordenadas de los nodos son:

- Origen ( $i$ ):  $(0, 0)$
- Objetivo ( $o$ ):  $(10, 0)$
- Nodo  $A$ :  $(2, 3)$
- Nodo  $B$ :  $(5, 0)$

Las conexiones (aristas) del grafo son únicamente:

- $(i, A)$
- $(i, B)$



#### Ejecución del algoritmo:

1. El mensaje está en  $u = i(0, 0)$ . Sus vecinos son  $A$  y  $B$ .
2. Calculamos las distancias de los vecinos al objetivo  $o(10, 0)$ :
  - $d(A, o) = \sqrt{(10 - 2)^2 + (0 - 3)^2} = \sqrt{64 + 9} = \sqrt{73} \approx 8.54$
  - $d(B, o) = \sqrt{(10 - 5)^2 + (0 - 0)^2} = \sqrt{25 + 0} = 5.0$
3. El algoritmo elige vorazmente el nodo  $B$  porque  $5.0 < 8.54$ .
4. El mensaje llega a  $u = B(5, 0)$ . El único vecino de  $B$  es  $i$ .
5. Comparamos las distancias al objetivo:
  - El nodo actual  $B$  está a distancia 5.0 de  $o$ .
  - Su único vecino  $i$  está a distancia 10.0 de  $o$ .
6. Como el nodo actual  $B$  es el más cercano al objetivo entre sus vecinos ( $5.0 < 10.0$ ), el algoritmo **termina el proceso y se detiene en  $B$** .

**Resultado:** El algoritmo falla. Se queda atrapado en el mínimo local  $B$  y el mensaje nunca se entrega a  $o$ , a pesar de que el grafo es conexo.